

Programmeringsparadigmer 2025

Session 7: Erklæring af typer og typeklasser

Opgaver til løsning og diskussion

Hans Hüttel

22. oktober 2025

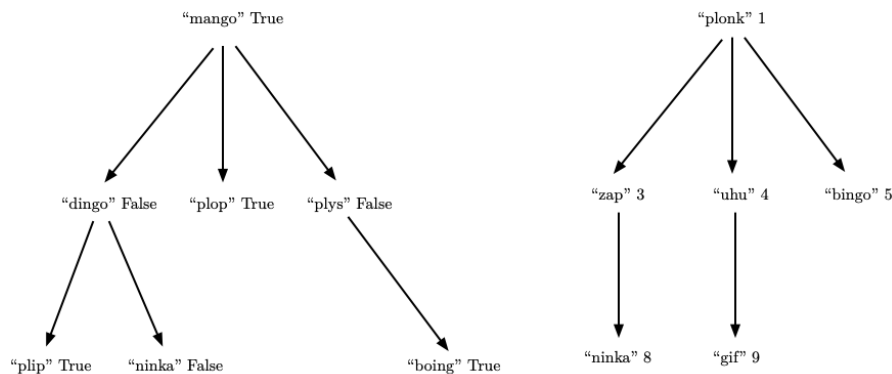
Opgaver, som vi helt sikkert vil tale om

1. **(10 minutter)** Definér en Haskell-datatype `Aexp` for aritmetiske udtryk med addition, multiplikation, tal og variable. Opbygningsreglerne i den abstrakte syntaks er

$$E ::= n \mid x \mid E_1 + E_2 \mid E_1 \cdot E_2$$

Antag, at variable x er strenge, og at tal n er heltal.

2. **(20 minutter)** Brug din Haskell-datatype fra den foregående opgave til at definere en funktion `eval`, som, når den får en term af typen `Aexp` og en tildeling `ass` af variable til tal, kan beregne værdien af udtrykket. Som eksempel, hvis vi har tildelingen $[x \mapsto 3, y \mapsto 4]$, skal `eval` kunne fortælle os, at værdien af $2 \cdot x + y$ er 10. *Hint:* En tildeling er en funktion. Hvordan kan du repræsentere den?
3. **(20 minutter)** Forfatterne til en ny encyklopædi om værdier af typen `a` repræsenterer deres information ved hjælp af en hierarkisk struktur, hvor hver post i encyklopædien er mærket med en nøgle, der er en streng, og en værdi af typen `a`. Poster kan have underposter. Figur 1 viser to encyklopædier.



Figur 1: To encyklopædier t_1 (venstre) og t_2 (højre)

Forfatterne til encyklopædien bad en berømt influencer om at definere en type `Encyclopedia` for encyklopædier. Influenceren skrev:

```
data Encyclopedia a = Leaf String a | Node1 String a (Encyclopedia a) | Node2 String a (Encyclopedia a) (Encyclopedia a) | Node3 String a (Encyclopedia a) (Encyclopedia a) (Encyclopedia a)
deriving Show
```

men klagede over Haskell's begrænsninger: "Jeg kan se af eksemplerne, at enhver post højst kan have tre underposter. Intet i Haskell tillader at skrive, at der kan være vilkårligt mange børn af samme type, så jeg er nødt til at skrive en meget klodset definition. *Haskell er ynkeligt!*"

- Kom med en bedre definition. Find ud af, hvad der er galt, og kom med en bedre løsning. Det er en god idé at læse opgaveteksten meget omhyggeligt og finde ud af, hvad den siger og ikke siger. Når du har kritiseret den eksisterende løsning, er det også en god idé *ikke at forsøge at reparere* den eksisterende løsning, men at starte forfra.

- Vis, hvordan du repræsenterer t_1 og t_2 i Figur 1 som værdier `t1` og `t2` af din type.
4. **(20 minutter)** En encyklopædi er *lagdelt*, hvis det gælder, at alle værdier på samme niveau i encyklopædien er større end værdierne på niveauerne ovenover. Som eksempel er t_2 i Figur 1 lagdelt, da 8 og 9 på niveau 3 er større end værdierne 3, 4 og 5 på niveau 2 – som igen er større end værdien 1 på niveau 1. Definér en funktion `layered`, der kan fortælle os, om en encyklopædi er lagdelt.
 5. **(20 minutter)** Fra lineær algebra ved vi, at et vektorrum med indre produkt er et rum, for hvilket operationerne vektorsum og indre produkt¹ er defineret. Givet to vektorer v_1 og v_2 , er summen $v_1 + v_2$ igen en vektor, og det indre produkt $v_1 \cdot v_2$ er et tal. *Vær opmærksom på:* I lineær algebra er der meget mere til definitionen af indre produkt rum end disse to krav, men i denne opgave skal du ignorere det. Antag også, at det indre produkt er et tal af typen `Int`.
 - Definér en typeklasse `InVector`, hvis instanser er typer, der kan betragtes som indre produkt rum, hvor vektorsum kaldes `&&&` og indre produkt kaldes `***`. *Hint:* Hvilket afsnit i bogen har du brug for her?
 - Find ud af, hvordan du erklærer `Bool` som en instans af `InVector`. *Hint:* Du skal finde en definition af vektorsum og indre produkt for sandhedsværdier.

Flere opgaver til løsning i dit eget tempo

Husk læringsmålene for denne session, når du løser opgaverne!

- a) Vi siger, at et binært træ er *balanceret*, hvis antallet af blade i ethvert venstre og højre undertræ højst adskiller sig med ét, hvor blade i sig selv er trivielt balancerede. Definér en funktion `balanced`, der kan fortælle os, om et binært træ er balanceret eller ej. *Hint:* Det er en god idé også at definere en funktion, der finder antallet af blade i et træ.
- b) Denne opgave refererer til afsnit 8.6 i bogen og de typer og funktioner, der er defineret der. To boolske udsagn p og q er *ækvivalente*, hvis de er sandt for de samme substitutioner, det vil sige, for enhver substitution s gælder det, at p er sandt under s hvis og kun hvis q er sandt under s . Definér en funktion `equiv`, hvor

```
equiv :: Prop -> Prop -> Bool
```

og sådan at `equiv p q` returnerer `True`, hvis p og q er ækvivalente, og `False` ellers. *Hint:* Hvad betyder "hvis og kun hvis" i logik?

- c) På side 106 er der en definition af `Expr`-datatypen for udtryk. Definér en højereordens funktion

```
foldexp :: (Int -> a) -> (a -> a -> a) -> Expr -> a
```

således at `foldexp f g` erstatter hver `Val`-konstruktør i et udtryk med funktionen `f` og hver `Add`-konstruktør med funktionen `g`. Brug derefter `foldexp f g` (med passende valg for `f` og `g`) til at definere en funktion `eval :: Expr -> Int`, der evaluerer et udtryk til en heltalværdi.

- d) Fuldfør følgende instansdeklaration:

```
instance Eq a => Eq (Maybe a) where
  ...
```

¹Også kaldet prikprodukt